Politechnika Krakowska
Wydział Inżynierii Elektrycznej i Komputerowej
Katedra Automatyki i Informatyki

Skrypt do laboratorium z przedmiotu:

Systemy Multimedialne

Tryb 15-bit i 16-bit

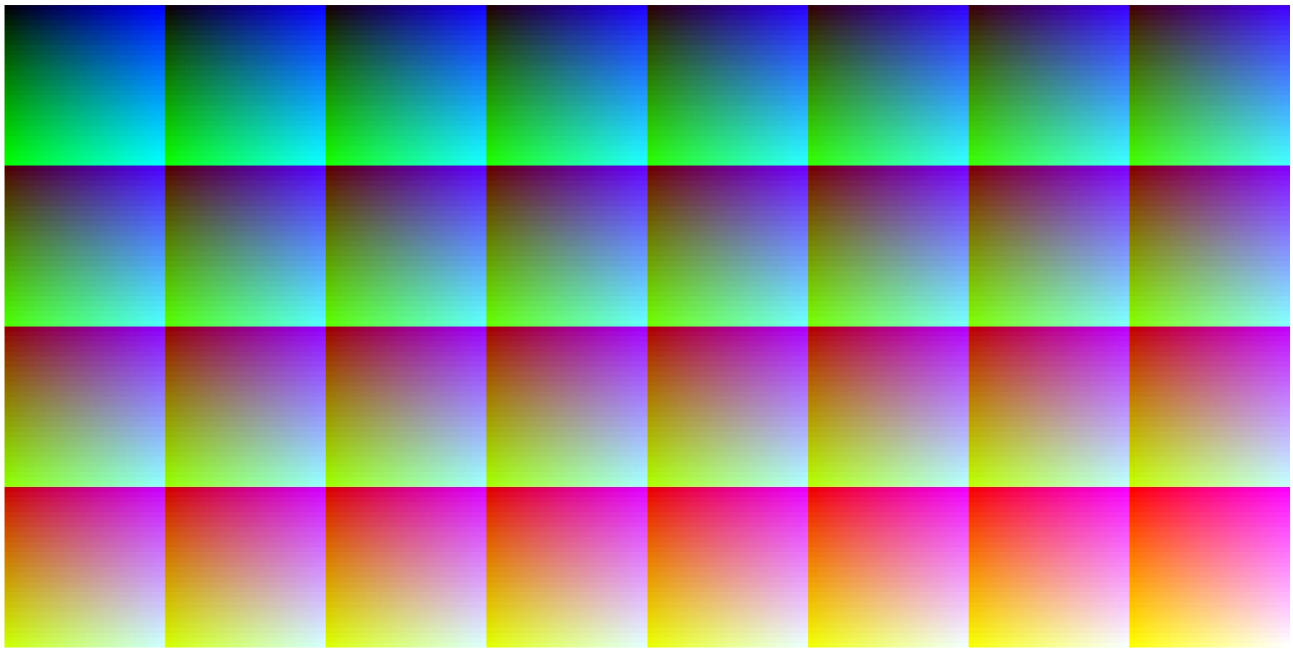
High color

16-bitowy kolor RGB (powszechnie nazwywany High color), to metoda przechowywania informacji o obrazie w pamięci komputera w taki sposób, że każdy piksel jest reprezentowany przez dwa bajty. Domyślnie kolor jest reprezentowany przez wszystkie 16 bitów, ale niektóre urządzenia obsługują także 15-bitowy kolor.

15-bit RGB

W przypadku koloru RGB w trybie 15-bitowym jeden z bitów jest ignorowany (lub używany jako forma kanału alfa), a pozostałe 15 bitów jest dzielone między składowe koloru czerwonego, zielonego i niebieskiego. Każda ze składowych RGB ma przypisanych po 5 bitów, co daje $2^5 = 32$ stany każdej składowej i pozwala na uzyskanie $2^5 \times 2^5 \times 2^5 = 32 \times 32 \times 32 =$ **32768** możliwych kolorów dla każdego piksela. Tryb ten czasami oznaczany jest jako **RGB555**.





Wśród popularnych zastosowań 15-bitowego trybu RGB należą następujące konsole:

- Super Nintendo Entertainment System (256 kolorów z 15-bitowego RGB)
- Nintendo Game Boy Color (56 kolorów z 15-bitowego RGB)
- Nintendo GameBoy Advance/SP/Micro (512 kolorów z 15-bitowego RGB)
- Nintendo DS (dla obrazu 2D)
- Neo Geo AES/Neo Geo CD (4096 kolorów z 15-bitowego RGB)
- Sega 32X (rozszerzenie do konsoli Sega Mega Drive/Genesis)
- Sony PlayStation (PSX wykorzystywało 24-bitową głębię kolorów do obliczeń, trybu 2D oraz wideo, ale tekstury stosowane do obiektów 3D miały maksymalną głębię kolorów 15-bitową).

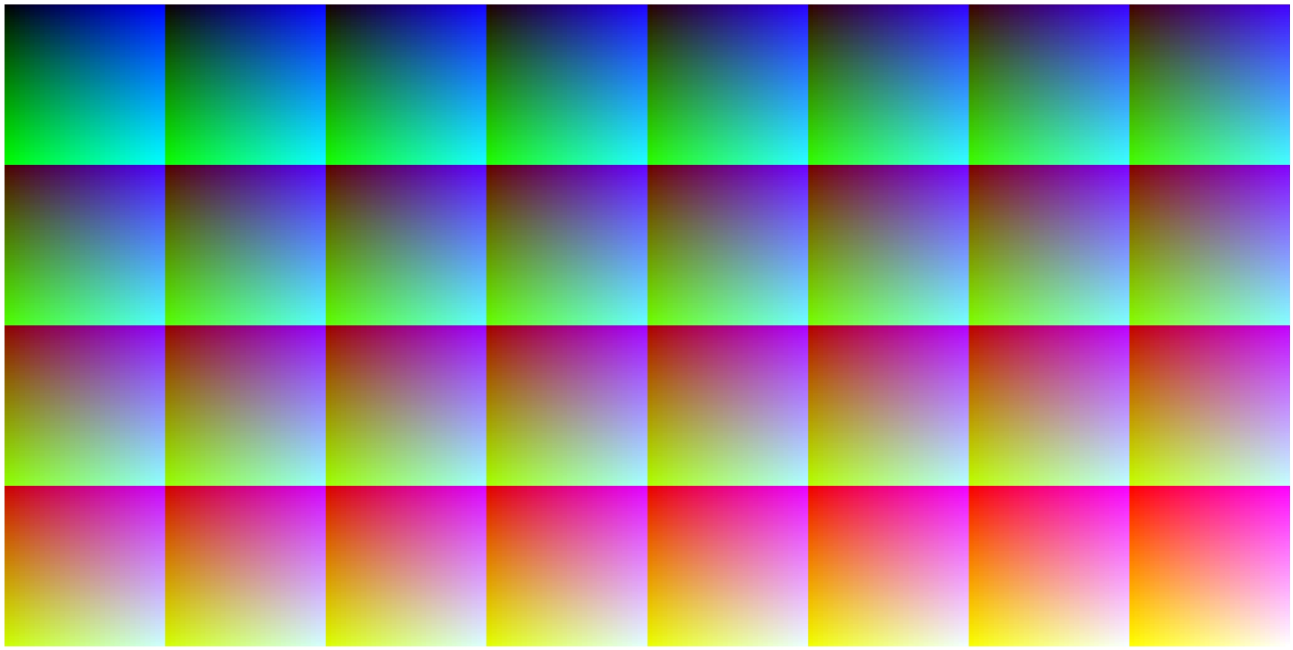
Popularne na początku lat ’90 układy graficzne Cirrus Logic wykorzystywały zapasowy bit w tak zwanych "mieszanych" trybach wideo: bity od 0 do 14 byłyby traktowane jako wartość RGB jak zostało to wcześniej wspomniane, podczas gdy w momencie ustawienia bitu 15, bity od 0 do 7 byłby interpretowane jako 8-bitowy indeks w palecie 256 kolorów (przy czym bity od 8 do 14 pozostawały w tym trybie nieużywane). Pozwalało to wyświetlanie (stosunkowo) wysokiej jakości kolorowych obrazów obok animowanych elementów ekranu z paletą.

16-bit RGB

Większość nowoczesnych systemów obsługuje tryb 16-bitowego RGB, którego czasami określa się mianem High color (podobnie jak 15-bitowe RGB), medium color lub tysięcy kolorów. Wykorzystuje paletę kolorów $2^5 \times 2^6 \times 2^5 = 32 \times 64 \times 32 =$ **65536** kolorów. Tryb ten czasami oznaczany jest jako **RGB565**. Zazwyczaj dla składowej czerwonej i niebieskiej przydzielane jest 5 bitów (po 32 stany) i 6 bitów dla składowej zielone (64 poziomy), ze względu na większą wrażliwość ludzkiego oka na ten kolor, co powoduje podwojenie poprzedniej 15-bitowej palety RGB.

Poniżej 16-bitowa paleta RGB wykorzystująca po 5 bitów dla składowej czerwonej i niebieskiej oraz 6 bitów dla składowej zielonej:





Tryb ten, mimo iż może wydawać się dużo bardziej korzystny w kontekście zapisu danych do pliku (łatwiej jest zapisać wartość piksela jako liczbę 16-bitową, tj. dwubajtową, niż jako liczbę 15-bitową), to sprawia drobne problemy w precyzji kodowania poszczególnych składowych. Różnica w precyzji domyślnej składowej zielonej względem pozostałych składowych może prowadzić do niewielkich rozbieżności w jej kodowaniu, np. gdy chcielibyśmy dokonać konwersji 24-bitowego RGB o wartości (40, 40, 40) na wartość 16 bitową. Wartość dziesiętna 40 w systemie binarnym to 00101000. Składowe czerwona i niebieska zajmą pięć najbardziej znaczących bitów i (korzystając z przesunięcia bitowego) będą miały wartość 00101, czyli 5 w skali od 0 do 31 (16,1%). Zielona składowa, z sześciobitową precyzją, będzie miał wartość binarną 001010, czyli 10 w skali od 0 do 63 (15,9%). Z tego powodu kolor RGB (40, 40, 40) będzie miał lekko purpurowy (magenta) odcień, gdy będzie wyświetlany w 16 bitach (oryginalna wartość 40 w skali 8-bitowej od 0 do 255, to odpowiednio 15,7%). Z kolei np 24-bitowa reprezentacja RGB o 14,1% poziomie szarości, tj. (36, 36, 36), ponieważ 36 jest reprezentowane jako 00100100 w systemie binarnym, byłaby dla składowej czerwonej i niebieskiej zakodowana jako 4/31 (12,9%), a dla zielonej jako 9/63 (14,3%), i będzie miała lekko zielony odcień.

Warto zwrócić uwagę na fakt iż nie wszystkie systemy korzystające z 16-bitowej głębi kolorów wykorzystują 16-bitową paletę RGB według podanej wcześniej reguły (565), np. komputer domowy Sharp X68000 lub konsola do gier wideo Neo Geo wykorzystują w rzeczywistości 15-bitową paletę RGB (po 5 bitów jest używanych dla składowych czerwonej, zielonej i niebieskiej), a ostatni bit określa mniej znaczącą intensywność lub luminancję.

Zadania do wykonania na zajęciach laboratoryjnych:

W zamieszczonym w instrukcji wprowadzeniu teoretycznym mamy informację o możliwości reprezentowania pikseli za pomocą dwóch bajtów trybach High color korzystających odpowiednio z 15 bitów na piksel (**RGB555**) oraz 16 bitów na piksel (**RGB565**). Są to modele które w większości przypadków w zupełności wystarczają do stosunkowo dobrej reprezentacji koloru.
Aby usprawnić korzystanie z tych modeli bardzo proszę o wykonanie poniższych zadań.

1. Zdefiniowanie i implementacja metod **setRGB555(xx, yy, r, g, b)** oraz **setRGB565(xx, yy, r, g, b)** pozwalającej na wstawienie na ekran piksela, którego wartości przekazywane są przez użytkownika z odpowiednią precyzją. Funkcje te muszą przekonwertować otrzymane wartości (r, g, b) na 24-bitowy model RGB i narysować na ekranie piksel korzystając ze standardowej funkcji setPixel(…).
2. Zdefiniowanie i implementacja metod **getRGB555(xx, yy)** i **getRGB565(xx, yy)** pozwalających na pobranie z ekranu wartości piksela z odpowiednią precyzją. Funkcje te muszą pobrać z ekranu wartość piksela za pomocą standardowej funkcji getPixel(…) i skwantyzować poszczególne składowe do odpowiedniej precyzji.

```
void setRGB555(int xx, int yy, Uint8 r, Uint8 g, Uint8 b);
void setRGB555(int xx, int yy, Uint16 rgb555);
void setRGB565(int xx, int yy, Uint8 r, Uint8 g, Uint8 b);
void setRGB565(int xx, int yy, Uint16 rgb565);
SDL_Color getRGB555(int xx, int yy);
Uint16 getRGB555_(int xx, int yy);
SDL_Color getRGB565(int xx, int yy);
Uint16 getRGB565_(int xx, int yy);
```

3. Wizualizacja trybów **RGB555** i **RGB565**. Proszę, korzystając ze stworzonych metod getRGB555, getRGB555_, getRGB565, getRGB565_ oraz setRGB55, setRGB565 zweryfikować poprawność ich działania, tj. proszę spróbować przerysować fragment obrazu (np. jeden z obrazków) i dokonać oceny poprawności wartości otrzymywanych podczas przeliczania modelu.